Statically Optimizing Memory Usage in Futhark

Philip Munksgaard

January 20, 2021

Contents

1 Motivation 1

1 Motivation

Futhark is a pure, functional array programming language. It has no notion of memory or resources, so it is entirely up to the compiler to transform the programs written by users into something that maps to the physical reality, including supporting a notion of memory.

```
1  let and_plus_one (xs: []i32): [](i32, i32) =
2    map (\x -> (x, x + 1)) xs
```

Figure 1: A simple futhark function.

As a short example, consider the `and_plus_one` function shown in figure 1. Each element of the input array is turned into a pair containing the original value and its successor. During compilation, the Futhark compiler will insert memory and turn it into the code shown in figure 2.

```
 1  -- xs_3955 : [impl _3954]i32@@xs_mem_3994->
 2  -- {base: [impl _3954]; contiguous: True; LMADs: [{offset: 0i64; strides: [1i64];
 3  --                                                 rotates: [0i64];
 4  --                                                 shape: [impl _3954];
 5  --                                                 permutation: [0];
 6  --                                                 monotonicity: [Inc]}]}
 7  fun {[impl _3954]i32@?0->
 8       {base: [impl _3954]; contiguous: True; LMADs: [{offset: 0i64;
 9                                                       strides: [1i64];
10                                                       rotates: [0i64];
11                                                       shape: [impl _3954];
12                                                       permutation: [0];
13                                                       monotonicity: [Inc]}]},
14       [impl _3954]i32@?1->
```

```
15       {base: [impl _3954]; contiguous: True; LMADs: [{offset: 0i64;
16                                                       strides: [1i64];
17                                                       rotates: [0i64];
18                                                       shape: [impl _3954];
19                                                       permutation: [0];
20                                                       monotonicity: [Inc]}]}}
21  and_plus_one_3876 (mem xs_mem_3994, i64 impl _3954, [impl _3954]i32 xs_3955) = {
22    let {i64 segmap_group_size_3988} =
23      get_size(segmap_group_size_3982, group_size)
24    let {i64 segmap_usable_groups_3989} =
25      sdiv_up64(impl _3954, segmap_group_size_3988)
26    let {i64 bytes_3996} = mul_nw64(4i64, impl _3954)
27    let {mem mem_3997} =
28      alloc(bytes_3996)
29    -- defunc_1_map_res_3990 : [impl _3954]i32@@mem_3997-> -- ref:and_plus_one_gpu_@
30    -- {base: [impl _3954]; contiguous: True; LMADs: [{offset: 0i64; strides: [1i64];
31    --                                                 rotates: [0i64];
32    --                                                 shape: [impl _3954];
33    --                                                 permutation: [0];
34    --                                                 monotonicity: [Inc]}]}
35    let {[impl _3954]i32 defunc_1_map_res_3990} =
36      segmap_thread
37      (#groups=segmap_usable_groups_3989; groupsize=segmap_group_size_3988)
38      (gtid_3979 < impl _3954) (~phys_tid_3980) : {i32} {
39        let {i32 x_3991} = xs_3955[gtid_3979]
40        let {i32 defunc_0_f_res_3992} = add32(1i32, x_3991)
41        return {returns defunc_0_f_res_3992}
42      }
43    in {xs_mem_3994, mem_3997, xs_3955, defunc_1_map_res_3990}
44  }
```

Figure 2: Futhark IR for the `and_plus_one` function from $\text{and}_{\text{plusone}}$

We can see the memory allocation inserted by the compiler on line 28, and the fact that the result of the map is stored in that memory block is shown on the next line.

When using Futhark, we are most often trying to make something run fast on a GPU. However, each allocation inserted by the Futhark compiler uses up valuable memory on the GPU.

References